

FRED TALKS

11th July 2026

CONTENTS

It's-a Me, Agentic AI

HAN, NOT SOLO · 2362 WORDS

Search has its own bitter lesson

DOUG TURNBULL · 1012 WORDS

How big are our embeddings now and why?

★♥☆ VICKI BOYKIS ★♥☆ · 1697 WORDS

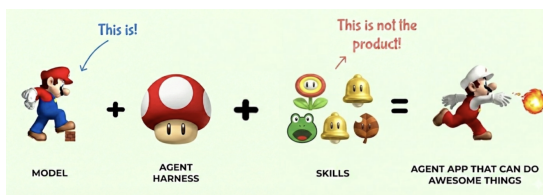
Toasting Marshmallows

XKCD.COM · 37 WORDS

It's-a Me, Agentic AI

HAN, NOT SOLO · 01 MAR 2026 · [SOURCE](#)

Agentic AI is a fairly recent development that combines reasoning ([OpenAI, 2024](#)) tool use ([Schick et al, 2023](#)) in the same AI model. But an agentic AI system is not just the model, but also the harness, the environments, the tools, rewards, evaluations, and all of the infrastructure to support it. In this post, let's use the top grossing franchise, Mario, to tell this story for understanding how agentic AI models are developed, how agent harnesses work, and how reinforcement learning ties everything together. If you survived World 8-4 as a kid, you already have the intuition for building agentic AI systems.



alt text

Small Mario is the **base pretrained model**. He's just come out of pretraining on a massive corpus of platform game physics. He can walk. He can jump. He can move left and right. These are his base capabilities, the raw knowledge compressed from the training data.

But Small Mario is fragile. One hit from a Goomba and he's dead. He can't break bricks. He can't take damage. He has potential, but he's not yet useful for anything beyond the most trivial tasks.

This is your base LLM fresh off pretraining. It has absorbed enormous amounts of knowledge, it can do next-token prediction, and it can sort-of follow instructions. But ask it to do anything real, reliably, in production, and it falls apart on the first obstacle. One Goomba and it's game over.

The Super Mushroom: Model Harness

Then Mario finds the **Super Mushroom**. He doubles in size. He can now break bricks. He can take a hit without dying. He goes from fragile to capable.

The Super Mushroom is the **model harness**, the entire infrastructure and post-training pipeline that transforms a base model into something production-ready. This includes:

- **Instruction tuning** (RLHF, DPO, etc.) to make the model actually follow directions
- **System prompts** that define personality and constraints
- **Safety guardrails** so it doesn't run off cliffs
- **Memory and context management** so it remembers where it's been
- **Tool-use training** so it knows power-ups exist and how to grab them

Without the Super Mushroom, Mario is a liability. With it, he's a platform for greatness. Similarly, without the model harness, a base LLM is a research artifact. With it, it's a product.

The Super Mushroom doesn't change WHO Mario is. It changes what he can SURVIVE. The model harness doesn't change the model's core knowledge. It changes what the model can handle in production.

Now here's where it gets interesting. Super Mario can pick up **power-ups** that give him entirely new capabilities:

Power-Up	Mario Ability	Agent Equivalent
Fire Flower	Throw fireballs to defeat enemies at range	Code execution - reach out and run code to solve problems the model can't solve with text alone
Frog Suit	Swim through water levels with ease	Web search - navigate and retrieve information from an environment the model wasn't trained on
Raccoon Leaf	Fly and reach otherwise inaccessible areas	File system access - explore and modify the codebase, reach files the model couldn't otherwise touch
Hammer Suit	Throw hammers, defeat almost anything	Shell/terminal access - the most powerful tool, can interact with the full system
Star	Temporary invincibility, blow through everything	Extended thinking/reasoning mode - brute force through complex problems at higher compute cost
Cape Feather	Sustained flight and spin attack	MCP servers - sustained, extensible access to external services and APIs

Each power-up doesn't replace Mario's core abilities. Mario still walks and jumps. The power-ups **extend** what he can do. A Fire Flower Mario can still jump on Goombas, but now he can also shoot fireballs at Piranha Plants hiding in pipes.

This is exactly how agent tools work. The LLM still does what LLMs do: reasoning, language understanding, and planning. Tools extend the model's reach into environments it can't operate in alone. An LLM can't execute Python by itself, just like Mario can't throw fireballs without a Fire Flower. But give it the right tool, and suddenly the problem space opens up.

And critically, **Mario has to learn WHEN to use each power-up**. Frog Suit is amazing in water levels, useless on land. Fire Flower is great against Goombas, pointless against Thwomps. The model needs to learn tool selection, knowing which tool to reach for in which context. This is one of the hardest parts of building agentic systems.

One power-up deserves special attention: the **Star**. When Mario grabs a Star, he becomes invincible. He plows through Goombas, Koopa Troopas, Piranha Plants, everything in his path just disintegrates. Nothing can stop him. This is not dissimilar to having an engineering manager who's really good at clearing organizational blockers for their engineers. The Goombas and Piranha Plants of bureaucracy, cross-team dependencies, access requests, and priority conflicts just melt away. Star power is temporary and expensive, but when you need to blast through a critical path, nothing else comes close.

The Mushroom Kingdom: Environments and Tasks

Now let's talk about the world Mario operates in. Every level in the Mushroom Kingdom is an **environment**, and every environment is composed of the same building blocks:

Level Element	Environment Equivalent
Bricks	Structured data, APIs, databases that can be broken open to reveal resources
? Blocks	Unknown information sources, sometimes containing exactly what you need
Pipes	Entry points to sub-tasks, function calls, or deeper exploration
Coins	Incremental progress signals, intermediate rewards
Goombas	Common obstacles, predictable errors, edge cases
Koopa Troopas	Recyclable obstacles, errors whose solutions can be reused as tools
Piranha Plants	Traps hiding in seemingly safe passages, hallucination triggers
Vines	Hidden paths to bonus areas, unexpected shortcuts

Clouds / Platforms

Abstractions and scaffolding that support traversal

Pits

Catastrophic failures, unrecoverable errors

Every level remixes these elements differently. World 1-1 is simple, a few Goombas, some bricks, a clear path to the flag. World 8-4 is a maze of pipes, hidden paths, and a boss fight with Bowser. Same building blocks, radically different difficulty.

Throughout each level, Mario interacts with the environment as **tool use**. He enters pipes to warp from one place to another, the equivalent of an API call that transports you to an entirely different context. He bumps ? Blocks from below to discover power-ups, new agent skills materializing from the environment when you know where to look. He breaks bricks to clear paths or reveal hidden rewards, structured data yielding its value when you apply force in the right direction. He stomps on a Koopa shell and kicks it forward, turning an obstacle into a projectile that clears a line of Goombas, repurposing error outputs as inputs to solve downstream problems. The environment isn't just a backdrop. It's a toolbox.

Having an environment is not enough. We need to define **what we want to achieve** from playing the game. These are the **tasks**.

Mario can play the same level with completely different objectives. Complete the level. Complete the level in the shortest amount of time. Get the highest score. Collect the most coins. Accumulate the most 1-up lives. Find all the hidden rewards. Stomp on every last Goomba. Each objective produces a fundamentally different play style from the same environment.

This is exactly the task definition problem in agentic AI. The same model operating in the same environment can be pointed at wildly different goals. “Summarize this codebase” and “refactor this codebase” use the same files, the same tools, the same context, but they require entirely different strategies. The task is what transforms an environment from a sandbox into a mission.

The Flagpole: Evaluations and Reward Modeling

At the end of every level, there's a **flagpole**. Mario jumps on it, pulls down the flag, and receives a reward. The higher he grabs the flag, the bigger the reward. Some levels end with a boss fight against Bowser, where the reward is freeing a Toad (or eventually, the Princess).

This is the **reward signal** in reinforcement learning. But here's the critical question: how do we actually **measure** how well the game was played? This is **evaluation**, or **reward modeling**, and it is where the machine learning engineering discipline really shines. (or research engineer, applied AI engineer.)

The evaluation could be the raw score. It could be the number of 1-ups gained. Coins collected. Goombas stomped. Time remaining. Different paths discovered. Most frequently, it is a **combination** of some or all of the above, weighted and balanced against each other. Do we reward Mario more for speed or for thoroughness? For survival or for aggression? For finding secrets or for staying on the critical path?

Evaluation Metric	Mario Measure	Agent Measure
Speed	Time remaining on the clock	Task completion latency
Score	Points accumulated	Overall output quality
Collection	Coins gathered	Information retrieved, resources used efficiently
Completeness	Hidden blocks found, secrets discovered	Edge cases handled, comprehensive coverage
Efficiency	Enemies defeated per life	Correct tool invocations per task
Exploration	Different paths taken	Novel approaches discovered

Designing these rewards is a strict machine learning engineering discipline. A poorly shaped reward function produces an agent that technically completes tasks but in degenerate ways, like a Mario speedrunner who clips through walls. Impressive, but not what we actually wanted. Reward hacking is the Goodhart’s Law of agentic AI: when a measure becomes a target, it ceases to be a good measure.

Here’s where the full picture comes together. How does Mario learn to complete levels?

Reinforcement learning.

Mario starts each level knowing nothing about its specific layout. He has to:

1. **Observe** the current state, what’s on screen, where the enemies are, what power-ups are available
2. **Decide** on an action based on his policy, jump, run, shoot, or wait
3. **Act** and receive feedback from the environment
4. **Update** his policy based on the outcome

This is the MDP (Markov Decision Process) loop. The same loop described in [Agents Are Workflows](#). The same loop that every agentic AI system runs:

The value of Mario's current state equals the expected immediate reward plus the discounted value of the next state. Should Mario jump NOW to get the coin, or wait and avoid the Goomba? The optimal policy balances immediate rewards against future outcomes.

Through repeated play (training episodes), Mario learns:

- Which obstacles can be jumped on vs. avoided
- When to use power-ups vs. save them
- Which pipes lead to shortcuts vs. dead ends
- How to handle boss fights

An agentic AI model goes through the same process. Through reinforcement learning (PPO, DPO, GRPO, or whatever the latest acronym is), the model learns:

- Which tools to invoke for which subtasks
- When to think longer vs. act immediately
- Which approaches work for which problem types
- How to decompose complex tasks into manageable steps

So who's actually making all of this work? Mario doesn't train himself.

To teach agent Mario to be really good at the game, we employ an **Machine Learning Engineer (MLE)**. The MLE is the game designer and coach rolled into one. They construct mock environments, set up the harnesses and tools, define the tasks that Mario needs to achieve, and most importantly, **design the reward function**. The MLE decides what "good" looks like. Do we reward Mario for speed? Thoroughness? Both? How much? This reward design is the single highest leverage decision in the entire pipeline. Get it right and Mario learns to play beautifully. Get it wrong and Mario learns to exploit glitches.

Once the MLE has designed the training setup, the **Machine Learning Systems Engineers (MLSys)** take over. They are the ones who actually run the show at scale. They set up the environment and agent Mario across hundreds to hundreds of thousands of environments, tasks and iterations. They manage the compute, the distributed training runs, the data pipelines. They collect the reasoning traces, the sequences of observations, actions, and outcomes from every single episode Mario plays. And from these traces, they run the reinforcement learning algorithms that allow agent Mario to learn from experience.

This is the unsexy but critical part. An MLE can design the most elegant reward function in the world, but without MLSys engineers standing up the infrastructure to run millions of training episodes and collect the resulting data, Mario never gets past World 1-1.

Worlds: Agent Frameworks and Domains

The Mushroom Kingdom isn’t one level. It’s organized into **Worlds**, each with a distinct theme and set of challenges:

World	Theme	Agent Domain
World 1	Grassland, basics	Simple text tasks, Q&A, summarization
World 2	Desert	Data analysis, structured environments
World 3	Water	Web and API navigation
World 4	Giant Land	Large-scale refactoring, migrations
World 5	Sky	Architecture and system design
World 6	Ice	Debugging in fragile or legacy environments
World 7	Pipe Maze	Complex multi-step workflows with dependencies
World 8	Bowser’s Castle	Full autonomous task completion with adversarial conditions

Agent frameworks are the **World Maps** - they organize how the agent navigates between levels (subtasks) and worlds (domains). Some frameworks are rigid, you follow the path laid out. Others let you pick your route. The best ones (like how Super Mario Bros. 3 introduced the overworld map) let the agent make strategic choices about which level to tackle next.

But remember: the model is the product. The World Map (framework) is useful scaffolding, but Mario does the actual playing. As models get stronger, they need less scaffolding. Today’s World 8 is tomorrow’s World 1.

Conclusion

The entire agentic AI stack maps to Mario with surprising fidelity:

Mario	Agentic AI
Small Mario	Base pretrained model
Super Mushroom	Model harness and post-training
Power-ups	Agent tools and skills
Star Power	Engineering manager clearing blockers

Levels	Task environments
Pipes, bricks, shells	Tool use within the environment
Objectives (speed, score, coins)	Task definitions
Flagpole / Boss	Reward signal and evaluation
Reward design	Reward modeling (ML engineering discipline)
Learning to play	Reinforcement learning
MLE as game designer	ML engineer designing tasks and rewards
MLSys running millions of episodes	ML infrastructure for training at scale
World Map	Agent framework
Multiplayer	Multi-agent systems
The next time someone asks you to explain agentic AI, ask them: have you played Mario?	

If they can understand that Small Mario needs a mushroom to survive, power-ups to be effective, and practice to master each level, they can understand agentic AI development.

Now go save Princess Peach.

References

```
@article{
  leehanchung,
  author = {Lee, Hanchung},
  title = {It's-a Me, Agentic AI},
  year = {2026},
  month = {02},
  day = {18},
  howpublished = {\url{https://leehanchung.github.io}},
  url = {https://leehanchung.github.io/blogs/2026/02/18/mario-agent}
}
```

Search has its own bitter lesson

DOUG TURNBULL · 22 JUN 2026 · [SOURCE](#)

An agent with grep does quite well at search.

It's shocking to search technologists. We want to cocoon the problem in technology. We care about algorithms, knowledge graphs, ranking: ever-and-ever smarter retrieval.

It turns out, though, you can play a bit of a trick. If you convince everyone to optimize content for your search engine, you'll have built the best search engine.

There's Sutton's bitter lesson about unleashing raw compute on a problem. But there's a different bitter lesson in search: algorithms matter less than convincing the world to optimize for your search engine.

Sure, technology acts as the trigger. In Web Search, Google shifted away from text relevance to prioritize authoritativeness (PageRank). Their success told the market: create content others want to link to. They reshaped the Web for good (or ill) around that incentive.

Google crowdsourced search quality to the Web. Claude Code outsources it to every developer.

Developers depend on agents to find documentation on the file system. We like that dopamine hit. That thrill of Claude Code knowing exactly how to reason about our code.

When agents make boneheaded mistakes, we stress. We frantically update documentation, we link to it from AGENTS.md, we add a new document, we place it in a useful location. We do anything - absolutely anything - to ensure agents know how to find and understand documentation.

Just as the SEO expert reads the tea leaves of Google's algorithm. The software developer writes to the LLM to make documentation findable + useful.

A search system with these incentives overcomes technologically sophisticated systems. For example, classic RAG rests on a foundation of technically sophisticated question answering. Answers often look like conversationally valid responses, but with little linkage back to questions it might answer:

Question:

| What is a runner test?

Answer:

| Verifies that small-dollar deposits and withdrawals can be executed against real test bank accounts during account reconciliation.

Yes that's literally an answer to the question.

But I wouldn't consider this "good documentation". It's not anchored to any concepts from the question. It's just an out of context bit of text.

If you wrote this for a coding agent, you'd probably organize the chunk to give hints about why it's useful.

```
docs/tests/runner.md
```

```
# Runner Tests
```

```
## Purpose
```

```
We need to ensure the system runs end to end
```

```
## Usage
```

```
Use this style of test sparingly. Unit and standard e2e tests should usually be preferred.
```

```
## What is it
```

```
A runner test actually interacts with real-life test bank accounts to ensure we can  
correctly deposit / withdraw money and that the full banking system works.
```

That's a far more descriptive piece of content to an LLM. Being encouraged to write *for the LLM itself* makes all the difference.

(As an aside, LLMs see candidate *answers* in their context before the user's question - so just treating RAG as question answering doesn't work).

What often matters isn't algorithms. It's programming the content author to create useful LLM context. That moves content discovery from random, disconnected factoids to documentation connecting information to problems to solve.

You learn that the trick to RAG isn't creating good question answering systems. Its writing better answers.

The enterprise search morass... is over?

Compare how coding agents program us to the eternal nightmare of your company's wiki.

When people write in the wiki, they do it for themselves. Nobody is incentivized to make information findable by anyone. No author sits obsessing how to ‘SEO’ the content for Confluence search.

If someone is ‘lucky’ enough to find your article, they can barely understand it. It’s written in your team’s jargon. And it’s probably out of date anyways.

For these reasons, I always tell people they’d do better to write a public blog article. Blogs assume zero context. They’re clearly timestamped as an artifact of a moment in time. To avoid shame or promote your career, you’re incentivized to create something useful, interesting, and findable.

Writing for a coding agent has the same property. You’re writing documentation for a targeted reader. You care to SEO your content for Claude Code + grep.

Enterprise search fails because there’s no feedback loop between authorship and findability. Coding agents solve that, finally giving some hope to organizing company information into findable knowledge.

The real outcome of coding agents isn’t just code generation. It’s closing the feedback loop between company context and the next piece of work being done. That old fable of Enterprise Search - improving workforce productivity by leveraging existing insight - can finally become true.

Feedback to content authors drives user relevance

Look around your system. Who creates the content? What’s in it for them to make content findable?

A recurring theme in my consulting looks something like this - a job search company notices spammy behavior from job posters. They keyword stuff every programming language in some hidden text element. The usual reaction: that’s spammy and we should ban them.

That’s too black and white. You have content creators willing to put in work. Shift their black-hat SEO over to an acceptable white-hat SEO. Document what’s acceptable. What’s considered malicious.

Focus on feedback. Make the content-optimization path rewarding. Give creators feedback+guidance during creation. LLMs help detect spam in real-time. They push content creators to author what’s useful not malicious.

Give content creators feedback. Once you've identified what 'good' is, give those content authors dopamine hits. Tell content creators what queries perform well for the content. Make it real-time, sticky, fun. Incentivize and reward behaviors you want - and punish those you don't.

Steer content creators towards the content you want. Don't hope algorithms can overcome drivel and slop.

Your search engine isn't a bundle of algorithms ranking for users. It's a relationship between authors and users. That bitter lesson puts grep above the smartest reranker and embedding model. Forget it at your peril.

Enjoy softwareDoug in training course form!

Starting May 18!

Signup here - <http://maven.com/softwareDoug/cheat-at-search>



I hope you join me at [Cheat at Search with Agents](#) to learn use agents in search. build better RAG and use LLMs in query understanding.

© 2026 SoftwareDoug LLC

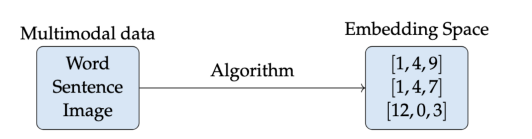
How big are our embeddings now and why?

★♥☆ VICKI BOYKIS ★♥☆ · 10 SEP 2025 · [SOURCE](#)

A few years ago, I wrote a [paper on embeddings](#). At the time, I wrote that 200-300 dimension embeddings were fairly common in industry, and that adding more dimensions during training would create diminishing returns for the effectiveness of your downstream tasks (classification, recommendation, semantic search, topic modeling, etc.)

I wrote the paper to be resilient to changes in the industry since it focuses on fundamentals and historical context rather than libraries or bleeding edge architectures, but this assumption about embedding size is now out of date and worth revisiting in the context of growing embedding dimensionality and embedding access patterns.

As a quick review, embeddings are compressed numerical representations of a variety of features (text, images, audio) that we can use for machine learning tasks like search, recommendations, RAG, and classification. The size of the embedding is how many features our item has.



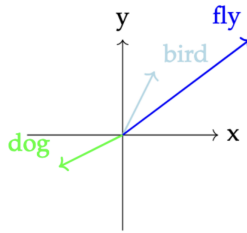
For example, let’s say we have two butterflies. We can compare them among many dimensions, including wingspan, wing color, number of antennae. Let’s say we have a 3-dimensional feature for a butterfly, it might look like this.

	wing_color	wing_span	antennae
butterfly_1	blue	10 cm	1
butterfly_2	red	20cm	2
butterfly_3	blue	15cm	1

Doing a visual vibe scan, we can eyeball the data and see that **butterfly_1** and **butterfly_3** are more similar to each other than to **butterfly_2** because their features are closer together.

But butterflies are 3-dimensional animals and some of these features are numerical. When we talk about embeddings in industry these days, we generally mean trying to understand the properties of text, so how does this concept work with words? We can’t directly compare words like “bird” and “butterfly” or “fly” in a given text but we can compare their numerical representations if we map them into the same shared space. We can see that “bird” and “flying” are more similar to each other than to “dog” through their numerical representations.

Intuitively, we know this is true, but how do we artificially create these relationships?

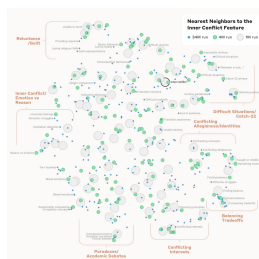


There are different ways of creating text embeddings from a given word, all of which rely on analyzing that word in relation to the words around it in a given corpus of data. We can use traditional count-based approaches like TF-IDF based on term frequency in documents, or statistical approaches like PCA or LSA.

With the advent of deep learning models, we started learning representations generated from models like Word2Vec that maximized the probability that a left-out word would be next to other given words in the training dataset.

When we learn the embedding representation of a given word using probabilistic models, we are comparing how similar these words are to other words. Each feature is not an explicit feature like “wing color”, but rather a vibe-based latent representation in the latent space that doesn’t have a clear explanation.

For example, one dimension might be “this word is an action word” or maybe “this word is related to other words about food”, but we generally don’t know exactly what the model thinks each feature represents. In fact, this is a fascinating area of study we are just starting to understand how these latent representations work through ideas like control vectors, a concept that Anthropic explored in the famous Golden Gate Claude paper.



When we train a model, embedding size is initialized as a hyperparameter before model training, and we iterate on the size depending on our downstream evaluations after training. Picking the right hyperparameter is (alchemy) a combination of art and science and depends on optimizing training throughput, final embedding storage size, and the performance of the embedding on your downstream task both qualitatively and wrt to latency of performing search on embeddings of different sizes.

Since previous generations of models were smaller and trained in-house, hyperparameters were usually not published by companies, and as such, there was no standard agreement on embedding size. We generally, as an industry, understood that somewhere around 300 dimensions for a given embedding model might be enough to compress all the nuance of a given textual dataset. 300 was the number of dimensions typically used by earlier models like Word2Vec and GloVE.

After the publication of the attention paper BERT was released in 2018. This model's architecture introduced embeddings of 768 dimensions. Although previous RNN and LSTM models had been trained on GPUs, BERT was one of the first larger embedding models to be trained on GPUs (and TPUs), which meant that GPU optimization now became increasingly important.

The key behind training Transformer models efficiently is the ability to efficiently move data onto GPU and parallelize their matrix multiplication operations between several pipelines, aka attention heads, where each attention head can focus on understanding and defining a different part of the embedding feature space. As such, the embedding needs to be able to be partitioned evenly between the number of attention heads.

BERT has 12 attention heads, so 768 dimensions was selected from a combination of trial and error and efficiently parallelizing computation to “attend” to different parts of the feature space, meaning that, in model training, each head operates on a 64-dimensional subspace of the original 768-dimensional input embedding. This itself comes from the Transformer paper, where each sub-embedding size per head is commonly chosen as 64.

As a result, many BERT-style models, and related model families, used 768 as a baseline for embedding dimensions. Although it had a much larger training dataset, GPT-2 also implemented 768. And, although CLIP uses an embedding size of **768** as derived from the Vision Transformer architecture that CLIP uses for its image encoder, and for consistency, the text encoder also uses this dimension size^[1].

Even though training cycles for BERT were fairly small (4 days for the original BERT model) compared to the months-long pretraining processes that LLMs require these days, it was still hard computationally to infer these embedding sizes, even with GPU optimizations. For BERT, for example,

Finding the most similar pair in a collection of 10,000 sentences req

So in 2019, UKP created and optimized a model, SBERT, focusing on sentence-level representations, which unlocked faster inference, and Minilm became the standard baseline model for document-level chunk embeddings at 384 dimensions and still performs extremely well for a number of tasks.

What else changed was the rise of the open-source HuggingFace platform where model artifacts could be shared. Previously, they were either hosted in undiscoverable repos in GitHub or locked away in scientific repositories missing metadata. The rise of HuggingFace as a platform and the transformers library as a centralized API into training and inference for PyTorch-based models unlocked a level of standardization: now, many more people could simply download and replicate the existing models instead of rebuilding arcane research code from scratch. This led to much more standard embedding and architecture sizes used both in academia and industry.

Although 768 held for a fairly long time, with upwards pressure from market competition on model sizes as the result of the release of GPT-2 (the backbone of ChatGPT), we started seeing standard embedding sizes increase.

Part of this was the fact that companies no longer had to infer their own embeddings. Previously, embeddings had been custom-learned in labs or in companies whose core competency was processing large amounts of information that could be culled in retrieval through search or recommendation.

But now, with the advent of ChatGPT and API-based model availability, embeddings became a commodity available with a GET request, and the most popular embeddings became OpenAI's, which used 1536 dimensions, in line with GPT-3, which used much more training data than any previous model. (570GB versus GPT-2 which was 40GB, and 96 attention heads.)

It used to be the case that people only learned or fine-tune their own embeddings, but now all of the major AI providers have their own hosted sets of embeddings. Particularly common ones are OpenAI, with Cohere and Nomic close behind. Google also recently released embeddings for Gemini, with both API and local versions being available.

In addition to standardization via HuggingFace and APIs, MTEB, which benchmarks embeddings publicly, has grown as a resource where you can compare embedding models.

If we look at MTEB, the embedding benchmark today, we can see embedding sizes anywhere from our classic 768 to 4096 and beyond (note all of them are neatly divisible by 2, in line with architectural constraints)

Rank (Res.)	Model	Zero-shot	Embedding D.
1	gemini-embedding-001	99%	3072
2	Qwen3-Embedding-8B	99%	4096
3	Qwen3-Embedding-4B	99%	2560
4	Qwen3-Embedding-0.6B	99%	1024
5	Llama-3.3-Embedding-Mistral	99%	4096
6	gpt4o-Qwen3-7B-Instruct	A NA	3584
7	multilingual-e5-large-instruct	99%	1024
8	gpt4o-Embedding-Mistral	98%	4096
9	text-multilingual-embedding-002	99%	768
10	Gemma-7B	99%	4096
11	Gemma-9B	99%	4096

Finally, another consideration has been the change in the landscape with vector databases, which used to be huge, are now becoming more commoditized features in software and platforms like Postgres, S3, and Elasticsearch, leading to less out of the box necessity in vector storage and performance tuning.

With the constant upward pressure on embedding sizes not limited by having to train models in-house, it's not clear where we'll slow down: Qwen-3, along with many others is already at 4096. It does appear that we are beginning to trim down on growth. OpenAI implemented a concept called matryoshka representation learning that trains embeddings with the “most important” concepts first, and additional embeddings adding incremental gains, meaning that an embedding learned in 1024 dimensions might be just as useful in 64, as long as the first 64 dimensions compress most of the information efficiently, and also making sure we re-normalize them. There are also research that indicates that not all embeddings are necessary in retrieval + search tasks, and that we can in fact, in some cases, truncate 50% of them anyway.

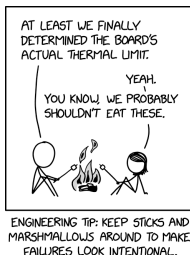
It's been fascinating to watch the rise of dimensionality and the constant struggle in tradeoffs between creating ever-larger models and then the need for those embedding sizes to perform at inference and storage time, aka continuously coming up against the age-old machine learning tradeoff of recall versus precision, and the engineering tradeoff of hardware limitations versus software versus business considerations.

As these architectures have matured and internal models have become inference points of public-facing paid APIs, embeddings have gone from a mysterious byproduct of internal machine learning systems in companies with lots of data to a commodity used across many AI-powered applications across numerous application stacks.

All in all, it's been so much fun watching this space evolve, even if it means it looks like I'll be having to update my paper over and over again.

Toasting Marshmallows

XKCD.COM · 11 JUL 2026 · [SOURCE](#)



We have two 'in case of fire, break glass' boxes, which you pick between based on how big of a problem the fire is. One has an extinguisher, and one has sticks, marshmallows, graham crackers, and chocolate.